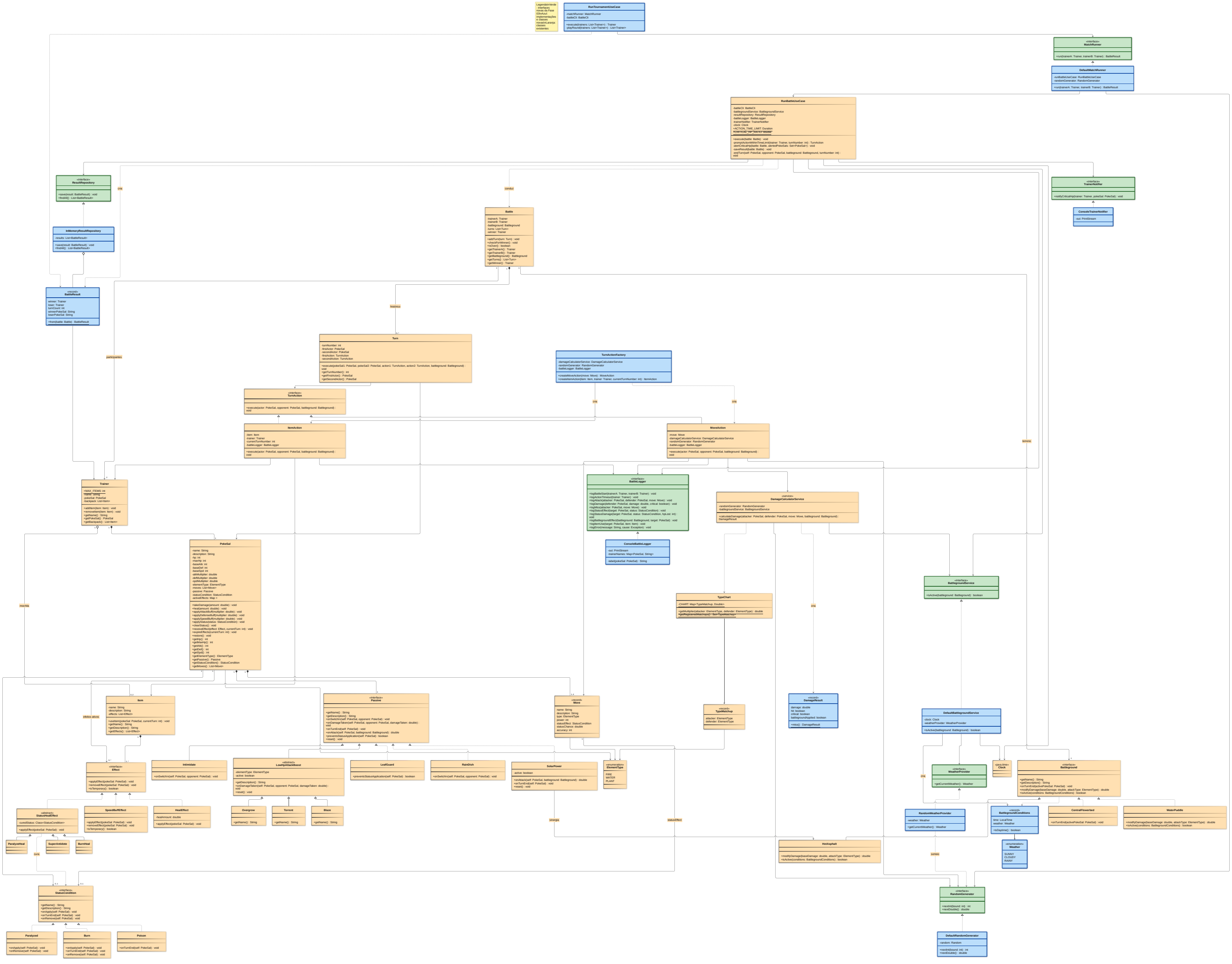
Diagrama de Classes - Sistema de Batalhas PokeSal (Fase 03)


Legenda\nVerde : interfaces novas da Fase 03\nAzul: implementações e classes novas\nLaranja: classes existentes
RunTournamentUseCase
-matchRunner: MatchRunner
-battleCli: BattleCli
+execute(trainers: List<Trainer>) : Trainer
-playRound(trainers: List<Trainer>) : List<Trainer>
«interface» MatchRunner
+run(trainerA: Trainer, trainerB: Trainer) : BattleResult
DefaultMatchRunner
-runBattleUseCase: RunBattleUseCase
-randomGenerator: RandomGenerator
+run(trainerA: Trainer, trainerB: Trainer) : BattleResult
RunBattleUseCase
-battleCli: BattleCli
-battlegroundService: BattlegroundService
-resultRepository: ResultRepository
-battleLogger: BattleLogger
-trainerNotifier: TrainerNotifier
-clock: Clock
+ACTION_TIME_LIMIT: Duration
+CRITICAL_HP_RATIO: double
+execute(battle: Battle) : void
-promptActionWithinTimeLimit(trainer: Trainer, turnNumber: int) : TurnAction
-alertCriticalHp(battle: Battle, alertedPokeSals: Set<PokeSal>) : void
-saveResult(battle: Battle) : void
-endTurn(self: PokeSal, opponent: PokeSal, battleground: Battleground, turnNumber: int) : void
«interface» TrainerNotifier
+notifyCriticalHp(trainer: Trainer, pokeSal: PokeSal) : void
ConsoleTrainerNotifier
-out: PrintStream
«interface» ResultRepository
+save(result: BattleResult) : void
+findAll() : List<BattleResult>
InMemoryResultRepository
-results: List<BattleResult>
+save(result: BattleResult) : void
+findAll() : List<BattleResult>
cria
«record» BattleResult
winner: Trainer
loser: Trainer
turnCount: int
winnerPokeSal: String
loserPokeSal: String
+from(battle: Battle) : BattleResult
participantes
conduz
Battle
-trainerA: Trainer
-trainerB: Trainer
-battleground: Battleground
-turns: List<Turn>
-winner: Trainer
+addTurn(turn: Turn) : void
+checkForWinner() : void
+isOver() : boolean
+getTrainerA() : Trainer
+getTrainerB() : Trainer
+getBattleground() : Battleground
+getTurns() : List<Turn>
+getWinner() : Trainer
1
1
1
histórico
0..*
Turn
-turnNumber: int
-firstActor: PokeSal
-secondActor: PokeSal
-firstAction: TurnAction
-secondAction: TurnAction
+execute(pokeSal1: PokeSal, pokeSal2: PokeSal, action1: TurnAction, action2: TurnAction, battleground: Battleground) : void
+getTurnNumber() : int
+getFirstActor() : PokeSal
+getSecondActor() : PokeSal
TurnActionFactory
-damageCalculatorService: DamageCalculatorService
-randomGenerator: RandomGenerator
-battleLogger: BattleLogger
+createMoveAction(move: Move) : MoveAction
+createItemAction(item: Item, trainer: Trainer, currentTurnNumber: int) : ItemAction
cria
cria
«interface» TurnAction
+execute(actor: PokeSal, opponent: PokeSal, battleground: Battleground) : void
ItemAction
-item: Item
-trainer: Trainer
-currentTurnNumber: int
-battleLogger: BattleLogger
+execute(actor: PokeSal, opponent: PokeSal, battleground: Battleground) : void
MoveAction
-move: Move
-damageCalculatorService: DamageCalculatorService
-randomGenerator: RandomGenerator
-battleLogger: BattleLogger
+execute(actor: PokeSal, opponent: PokeSal, battleground: Battleground) : void
terreno
«interface» BattleLogger
+logBattleStart(trainerA: Trainer, trainerB: Trainer) : void
+logActionTimeout(trainer: Trainer) : void
+logAttack(attacker: PokeSal, defender: PokeSal, move: Move) : void
+logDamage(defender: PokeSal, damage: double, critical: boolean) : void
+logMiss(attacker: PokeSal, move: Move) : void
+logStatusEffect(target: PokeSal, status: StatusCondition) : void
+logStatusDamage(target: PokeSal, status: StatusCondition, hpLost: int) : void
+logBattlegroundEffect(battleground: Battleground, target: PokeSal) : void
+logItemUse(target: PokeSal, item: Item) : void
+logError(message: String, cause: Exception) : void
ConsoleBattleLogger
-out: PrintStream
-trainerNames: Map<PokeSal, String>
-label(pokeSal: PokeSal) : String
«service» DamageCalculatorService
-randomGenerator: RandomGenerator
-battlegroundService: BattlegroundService
+calculateDamage(attacker: PokeSal, defender: PokeSal, move: Move, battleground: Battleground) : DamageResult
Trainer
2
+MAX_ITEMS: int
-name: String
-pokeSal: PokeSal
-backpack: List<Item>
+addItem(item: Item) : void
+removeItem(item: Item) : void
+getName() : String
+getPokeSal() : PokeSal
+getBackpack() : List<Item>
1
1
mochila
1
PokeSal
-name: String
-description: String
-hp: int
-maxHp: int
-baseAtk: int
-baseDef: int
-baseSpd: int
-atkMultiplier: double
-defMultiplier: double
-spdMultiplier: double
-elementType: ElementType
-moves: List<Move>
-passive: Passive
-statusCondition: StatusCondition
-activeEffects: Map >
+takeDamage(amount: double) : void
+heal(amount: double) : void
+applyAttackBuff(multiplier: double) : void
+applyDefenseBuff(multiplier: double) : void
+applySpeedBuff(multiplier: double) : void
+applyStatus(status: StatusCondition) : void
+clearStatus() : void
+receiveEffect(effect: Effect, currentTurn: int) : void
+expireEffects(currentTurn: int) : void
+restore() : void
+getHp() : int
+getMaxHp() : int
+getAtk() : int
+getDef() : int
+getSpd() : int
+getElementType() : ElementType
+getPassive() : Passive
+getStatusCondition() : StatusCondition
+getMoves() : List<Move>
1
1
1
1
TypeChart
-CHART: Map<TypeMatchup, Double>
+getMultiplier(attacker: ElementType, defender: ElementType) : double
+getRegisteredMatchups() : Set<TypeMatchup>
cria
«record» TypeMatchup
attacker: ElementType
defender: ElementType
«record» DamageResult
damage: double
hit: boolean
critical: boolean
battlegroundApplied: boolean
+miss() : DamageResult
«interface» BattlegroundService
+isActive(battleground: Battleground) : boolean
DefaultBattlegroundService
-clock: Clock
-weatherProvider: WeatherProvider
+isActive(battleground: Battleground) : boolean
0..2
Item
-name: String
-description: String
-effects: List<Effect>
+useItem(pokeSal: PokeSal, currentTurn: int) : void
+getName() : String
+getDescription() : String
+getEffects() : List<Effect>
efeitos ativos
1
0..*
1..*
«interface» Effect
+applyEffect(pokeSal: PokeSal) : void
+removeEffect(pokeSal: PokeSal) : void
+isTemporary() : boolean
1
«interface» Passive
+getName() : String
+getDescription() : String
+onSwitchIn(self: PokeSal, opponent: PokeSal) : void
+onDamageTaken(self: PokeSal, opponent: PokeSal, damageTaken: double) : void
+onTurnEnd(self: PokeSal) : void
+onAttack(self: PokeSal, battleground: Battleground) : double
+preventsStatusApplication(self: PokeSal) : boolean
+reset() : void
1..4
«record» Move
name: String
description: String
type: ElementType
power: int
statusEffect: StatusCondition
statusChance: double
accuracy: int
1
Intimidate
+onSwitchIn(self: PokeSal, opponent: PokeSal) : void
«abstract» LowHpAttackBoost
-elementType: ElementType
-active: boolean
+getDescription() : String
+onDamageTaken(self: PokeSal, opponent: PokeSal, damageTaken: double) : void
+reset() : void
LeafGuard
+preventsStatusApplication(self: PokeSal) : boolean
RainDish
+onSwitchIn(self: PokeSal, opponent: PokeSal) : void
SolarPower
-active: boolean
+onAttack(self: PokeSal, battleground: Battleground) : double
+onTurnEnd(self: PokeSal) : void
+reset() : void
«enumeration» ElementType
FIRE
WATER
PLANT
cria
«interface» WeatherProvider
+getCurrentWeather() : Weather
«java.time» Clock
1
«interface» Battleground
+getName() : String
+getDescription() : String
+onTurnEnd(activePokeSal: PokeSal) : void
+modifyDamage(baseDamage: double, attackType: ElementType) : double
+isActive(conditions: BattlegroundConditions) : boolean
«abstract» StatusHealEffect
-curedStatus: Class<StatusCondition>
+applyEffect(pokeSal: PokeSal) : void
SpeedBuffEffect
+applyEffect(pokeSal: PokeSal) : void
+removeEffect(pokeSal: PokeSal) : void
+isTemporary() : boolean
HealEffect
-healAmount: double
+applyEffect(pokeSal: PokeSal) : void
Overgrow
+getName() : String
Torrent
+getName() : String
Blaze
+getName() : String
sinergia
statusEffect
RandomWeatherProvider
-weather: Weather
+getCurrentWeather() : Weather
«record» BattlegroundConditions
time: LocalTime
weather: Weather
+isDaytime() : boolean
CentralFlowerbed
+onTurnEnd(activePokeSal: PokeSal) : void
WaterPuddle
+modifyDamage(baseDamage: double, attackType: ElementType) : double
+isActive(conditions: BattlegroundConditions) : boolean
ParalyzeHeal
SuperAntidote
BurnHeal
cura
HotAsphalt
+modifyDamage(baseDamage: double, attackType: ElementType) : double
+isActive(conditions: BattlegroundConditions) : boolean
«enumeration» Weather
SUNNY
CLOUDY
RAINY
sorteio
0..1
0..1
«interface» StatusCondition
+getName() : String
+getDescription() : String
+onApply(self: PokeSal) : void
+onTurnEnd(self: PokeSal) : void
+onRemove(self: PokeSal) : void
«interface» RandomGenerator
+nextInt(bound: int) : int
+nextDouble() : double
Paralyzed
+onApply(self: PokeSal) : void
+onRemove(self: PokeSal) : void
Burn
+onApply(self: PokeSal) : void
+onTurnEnd(self: PokeSal) : void
+onRemove(self: PokeSal) : void
Poison
+onTurnEnd(self: PokeSal) : void
DefaultRandomGenerator
-random: Random
+nextInt(bound: int) : int
+nextDouble() : double